

6 - BOB. JAVASCRIPT DASTURIY TILINING ASOSLARI

§ 6.1. Web – sahifani boshqarish tili - JavaScript

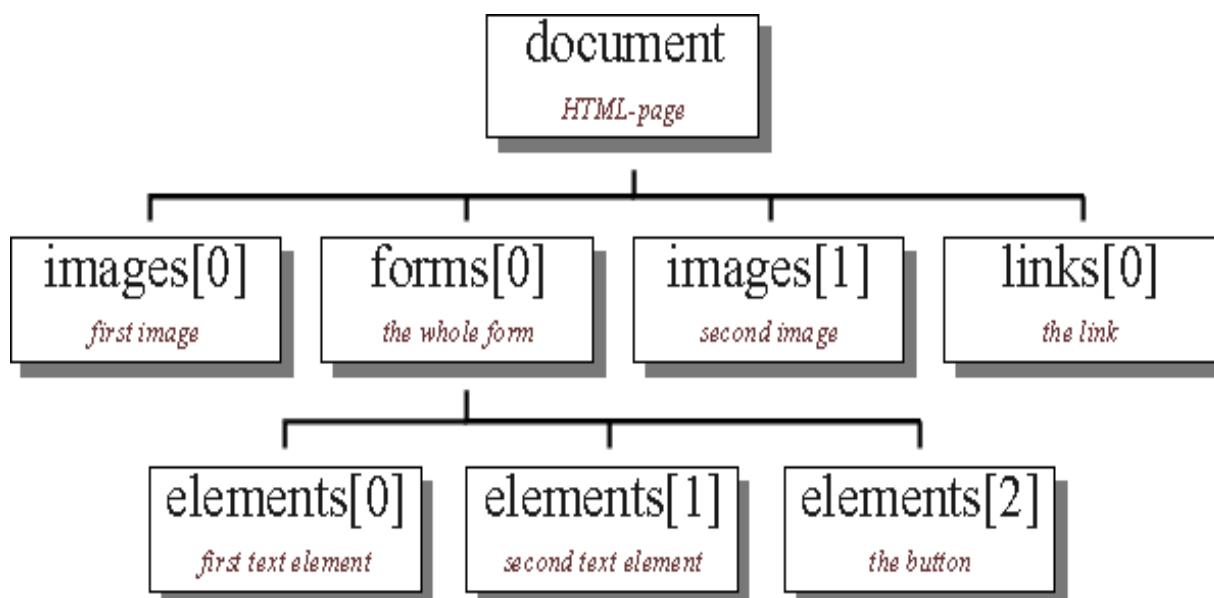
Web-sahifalarni klient tomonidan yoki server tomonidan boshqarish mumkin. 1995 yili Netscape kompaniyasi mutaxassislari JavaScript dasturiy tilini ishlab chiqib, sahifani klient tomonidan boshqarish mexanizmini hosil qildilar. Shunday qilib, JavaScript – bu gipermatnli web – sahifani ko‘rish stsenariysini klient tomonidan boshqarish tilidir[15]. Java Scriptdasturiy tili yordamida web sahifada interaktiv elementlarni hosil qilish mumkin. Java Scriptning asosiy g‘oyasi HTML-dasturida ifodalangan atributlarning qiymatini o‘zgartirish imkoniyatini hosil qilishdir. Bunda sahifani qayta yuklash vujudga kelmaydi. Amalda bu quyidagi jarayonlarda aks etadi. Masalan: sahifa fonining rangini o‘zgartirish, tasvirning elementlarini o‘zgartirish, yangi muloqot oynasini ochish hamda ogohlantirish berish kabidir. Har bir HTML hujjat ob’ekt bo‘lishi mumkin va u quyidagicha harakterlanadi:

- xususiyatlar;
- metodlar(usullar-dasturlar-funktsiyalar);
- xodisalar.

Ob’ektiv modelni Web sahifa va brouzerlarning aloqa usullari ko‘rinishida aks ettirish mumkin. Ob’ektiv model – bu ob’ektlar, usullar, xususiyatlar va boshqa elementlarning brouzer dasturiy ta’minotida namoyon bo‘lishidir. Ob’ekt – bu JavaScriptdasturiy tilining berilgan elementlari qiymatining turidir. Bir xil turdagi bir xil xususiyatga ega bo‘lgan, bir xil usulga ega bo‘lgan, bir xil xodisaga ega bo‘lgan ob’ektlar bitta sinfga birlashadi (klass). Sinflar – bu ob’ektlarning mumkin bo‘lgan taxriridir. Ob’ektlarning o‘zi brouzer programmasi ishga tushgandan so‘ng hosil bo‘ladi. Ob’ektga mo‘ljallangan dasturlash tili ob’ektlar sinfining pog‘onalashishini tavsiflaydi. JavaScript dasturiy tilida web-sahifaning hamma elementlari pog‘onalangan strukturaga joylashadi. Har bir element ob’ekt sifatida ko‘rinadi. Java Script tilida ob’ektlar Window sinfidan boshlab pog‘onalanadi. Har qanday element

ob'ekt ko'rinishida beriladi va shunday element o'z xususiyatlariga va metodlariga egadir. Biror ob'ektga yoki uning xususiyatiga murojaat qilish uchun ushbu elementning nomi o'zidan yuqorigi ob'ektning nomi bilan qo'shilib ko'rsatiladi[15,16]. JavaScript tili ob'ektga mo'ljallangan dasturiy tillarning klassik vakili hisoblanmaydi. Unda avlodlik (nasledovaniya) va polimorfizm mavjud emas. Dastur tuzuvchi shaxsiy ob'ektlar sinfini function operatori yordamida aniqlashi mumkin. Lekin ko'pincha standart ob'ektlardan, ularning konstruktsiyalaridan foydalanadi, sinflar destruktorini qo'llamaydi. Bu xolat shu bilan izohlanadiki, JavaScript dasturlarining xarakatlanish doirasi odatda joriy (tekushiy) muloqot oynasining chegarasidan chiqmaydi.

JavaScript dasturiy tilining umumiy tushunchasiga asosan brouzerning muloqot oynasi bu WINDOW ob'ektidir. Ushbu ob'ekt ham o'z navbatida ba'zi elementlarga egadir, masalan, holatni ko'rish qatori mavjud. Muloqot oynasining ichida HTML hujjatni yoki biror boshqa faylni joylashtirish mumkin. Bunday sahifa DOCUMENT ob'ekti bo'lib xizmat qiladi. Bu holat JavaScript dasturiy tilida shuni bildiradiki, DOCUMENT ob'ekti ayni vaqtda yuklangan HTML-hujjatdir.



6.1-rasm. HTML-sahifa ob'ektlarining ierarxiyasi⁵⁰.

DOCUMENT ob'ektining xususiyatlariga, masalan, web-sahifa fonining rangi kiradi. HTML-hujjatning barcha ob'ektlari DOCUMENT ob'ektining xususiyatlari

⁵⁰Rob Larsen. Beginning HTML & CSS. Wiley Publishing, Inc. Indianapolis, Indiana, 2013.

hisoblanadi. HTML-hujjatning ob'ektlariga misol qilib gipermurojaatni yoki qiymat bilan to'ldirilgan shaklni ko'rsatish mumkin. 6.1-rasmda HTML-sahifa tashkil qilgan ob'ektlar ierarxiyasi ko'rsatilgan. Rasmdan ko'rinib turibdiki, ierarxik strukturaning har bir ob'ekti o'zining nomiga ega. HTML-sahifaning birinchi rasm ob'ektiga qanday murojaat qilish mumkin. Murojaat qilishni eng yuqori cho'qqidagi ob'ektdan boshlash kerak. Bunday strukturaning birinchi ob'ekti DOCUMENT deyiladi. Sahifadagi birinchi rasm IMAGES[0] ob'ekti sifatida namoyon bo'ladi. Bu shuni bildiradiki, JavaScript dasturiy tilida ushbu ob'ektga murojaat qilish quyidagicha yoziladi: document.images[0].

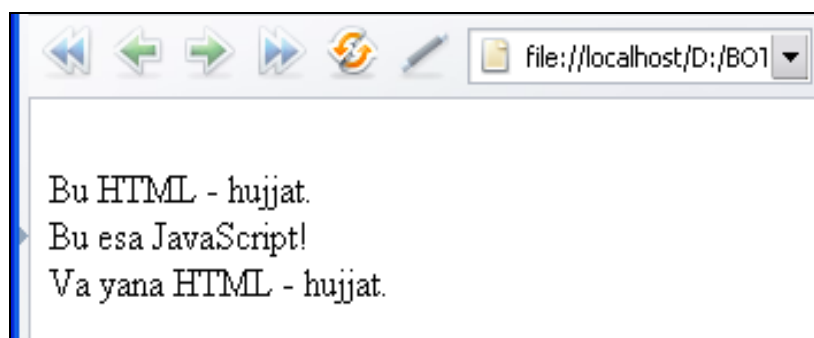
Foydalanuvchi shaklning birinchi elementiga qanday matn kiritganini bilish uchun avval ushbu elementga qanday murojaat qilish mumkinligini bilish kerak.

§ 6.2. JavaScriptniHTML – dasturda joylashtirish

JavaScript tilida yozilgan skriptlarni ishga tushirish uchun JavaScript - ishlovchi browser kerak bo'ladi. Java Script kodini HTML sahifasida joylashtirishni quyidagi misolda ko'ramiz:

```
<html><body> Bu HTML - hujjat.<br>  
<script language="JavaScript">  
document.write("Buesa JavaScript!")  
</script><br>VA yana HTML - hujjat. </body></html>
```

Ushbu misolda eng sodda HTML faylni yozib va uni browserga kiritiladi. Natijada 3 satrli fayl hosil bo'ladi (6.2-rasm).



6.2-rasm Java Script kodini HTML sahifasida joylashtirish dasturi natijasi⁵¹.

⁵¹ДроговВ.А. JAVA SCRIPT в Web – дизайне. – СПб.: «БХВ-Петер-бург», 2002.- 880 с.

§ 6.3. Xususiyatlar, usullar, hodisalar

Ko‘pchilik HTML – hujjatlar atributlarga egadir. Masalan, <A> tegi HREF atributiga ega : tdiu_info_teh.

Agar <A> tegining konteynerini ob’ekt sifatida olsak, u holda HREF atributi ushbu konteynerning xususiyatini belgilaydi. Dasturchi atribut qiymatini va ob’ektning xususiyatini o‘zgartirishi mumkin[12,15]:

```
document.links[0].href="info_teh.html".
```

Hamma atributlarning ham qiymatlarini o‘zgartirish mumkin emas. Masalan, grafik tasvirning balandligi va kengligi web – sahifa ishga tushganda uning aks etgan vaqtidagi o‘lchamiga asosan belgilanadi. Keyingi tasvirlar ham shu o‘lchamda aks etadi. Shuni aytib o‘tish zarurki. JavaScript dasturiy tilida grafik tasvirlar xususiyatlariga ko‘ra ob’ektlarga bo‘lingan. JavaScript dasturiy tilida ob’ektning metodlari uning xususiyatini o‘zgartirish funksiyasini aniqlaydi. Masalan, “document” ob’ekti bilan open(), write(), close() metodlari bog‘liqdir. Bu metodlar HTML hujjatining qiymatini va xususiyatini o‘zgartirish imkoniyatiga egadir.

```
<html><head><title> metodlar </title></head>
```

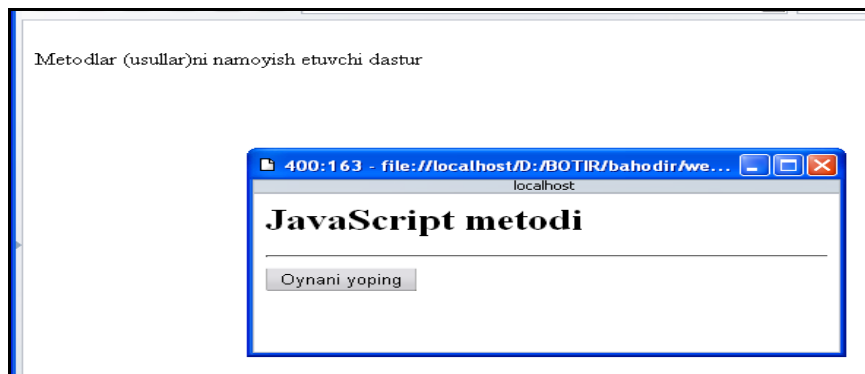
```
<body><h3>Metodlar (usullar)ni namoyish etuvchi dastur</h3>
```

```
<script language="JavaScript">
```

```
{ id=window.open("", "example", "width=400, height=150");  
  id.focus(); id.document.open();  
  id.document.write("<H1>JavaScript metodi</H1>");  
  id.document.write("<HR><FORM>");  
  id.document.write("<INPUT TYPE=button VALUE='Oynani yoping' ");  
  id.document.write("onClick='window.opener.focus(); window.close();>");  
  id.document.close();}
```

```
</script></body></html>
```

Ushbu dasturning bajarilishi natijasida quyidagi ko‘rinishdagi (6.3-rasm) web-sahifa namoyon bo‘ladi:



6.3-rasm. JavaScript kodini HTML sahifasida joylashtirishnatijasi⁵².

Ob'ektlar xususiyatlar va metodlardan tashqari hodisalar bilan ham xarakterlanadi. Hodisalar va ishlov beruvchi hodisalar JavaScriptdasturiy tilining muhim qismi hisoblanadi. Hodisalar foydalanuvchining turli hil harakatlari bilan yuzaga keladi. Ba'zi knopkalar bosilganda **"Click"** hodisasi sodir bo'ladi. Agar sichqoncha ko'rsatkichi gipermatn sahifasiga yo'naltirilsa **MouseOver** hodisasi ro'y beradi. Hodisalarning har xil turlari mavjud. JavaScript tilida dasturlashning mohiyati yuqoridagi hodisalarni taxlil qiluvchi va uning natijasida qaror qabul qiliuvchi dasturlarni yozishdan iboratligidir. JavaScript tilidagi dasturlar hodisalarning ro'y berishi natijasida harakatlanishi mumkin va bu maxsus dasturlar hodisalarni taxlil qiluvchilar (yoki hodisalarga ishlov beruvchilar deyiladi) (obrabotchiki sobyitiy) yordamida amalga oshiriladi. Masalan, knopka bosilganda muloqot oynasi hosil bo'lishi mumkin. Bu esa muloqot oynasini hosil qilish uchun **"Click"** hodisasi ro'y berishi kerakligini anglatadi. Bu holda qo'llaniluvchi dastur **onClick** deb nomlanadi va ushbu hodisa sodir bo'lganda kompyuterga qanday qaror qabul qilish zarurligini ma'lum qiladi[11,15]. Masalan, button ob'ekti bilan (button tipining INPUT konteyneri –"Knopka") click hodisasi sodir bo'lishi mumkin va foydalanuvchi knopkani bosishi mumkin. Buning uchun INPUT konteynerining click taxlil qilish hodisasi onClick atributi bilan kengaytirilgan. Ma'no sifatida bu atributning ishlov beruvchi hodisalar programmasi ko'rsatiladi va buni JavaScript tilida HTML – hujjatning muallifi yozishi kerak bo'ladi:

<INPUT TYPE=button VALUE="Bosing" onClick="window.alert('Iltimos,

⁵²Дрогов В.А. JAVA SCRIPT в Web – дизайне. – СПб.: «БХВ-Петер-бург», 2002.- 880 с.

yana bir bor bosing');">

Ishlov beruvchi hodisalar aynan o'sha konteyner bilan bog'liq bo'lgan hodisalarda ko'rsatiladi. Masalan: BODY konteyneri butun hujjatlarning xususiyatini aniqlaydi, shuning uchun ishlov beruvchi hodisalar barcha xujjatlar yuklanishining tugallanishi ushbu konteynerning onLoad atributi sifatida ko'rsatiladi.

§ 6.4. JavaScript dasturiy tili operatorlari

JavaScript tilining quyidagi operatorlarini ko'rib chiqamiz: var; {...}; if; while; for; for ... in; break; continue; return. Bu ro'yxat to'liq emas.

Operator var

var operatori o'zgaruvchini ifodalash uchun xizmat qiladi. Shuningdek, o'zgaruvchi har qanday hal qilingan ma'lumotlar turlari qiymatini qabul qilishi mumkin. Amalda ko'pincha var qo'llanilmaydi. O'zgaruvchi turga muvofiq o'zlashtirishning oson yo'li bilan vujudga keltiriladi:

```
var a;
```

```
var a=10;
```

```
var a = new Array();
```

```
var a = new Image();
```

Yuqorida keltiritgan barcha misollarda var to'g'ri foydalanilgan va JavaScript dasturida qo'llanilishi mumkin. O'zgaruvchi foydalanilayotgan harakat sohasida o'zgaruvchi blok orqali aniqlanadi. O'zgaruvchining maksimal harakat sohasi – sahifadir.

Figurali qavslar {...}

JavaScript bir nechta ifodani bitta ifodaga birlashtirish imkonini beradi. Bunday ifoda blokli ifoda yoki blok deb ataladi. Buning uchun kerakli kod figurali qavs {} ichiga olinadi [12,15].

```
a = 11; {b = "12"; c = a-b;}
```

Bloklardan murakkab ifodalar, xususan, shartli operatorlarga ega ifodalar tuzish uchun foydalaniladi.

Shartli operator if

Shartli operator dasturni tarmoqlanishi uchun ba'zi mantiqiy shartlarda ishlatiladi. Umumiy sintaksis:

```
if (mantiqiy ifoda) operator1; [else operator2;]
```

Mantiqiy ifoda — bu shunday ifodaki, u true yoki false qiymatini qabul qiladi. Agar u teng bo'lsa, true 1 operator amalga oshiriladi. Kvadratik qavslarda tuzilmaviy operator if- asosiy tarmoqni hisoblash alternativi shart emas:

```
if (navigator.appName=="Netscape")
    {window.location.href="http://ziyo.edu.uz/netscape.html"; }
else {window.location.href= "http://ziyo.edu.uz/explorer.html"; }
```

Tarmoqlash operatori muayyan hodisa sodir bo'lganida kodning muayyan qismini bajarish imkonini beradi. O'zgaruvchiga muayyan ifodaning berilishi yoki muayyan ifodani hisoblashning u yoki bu tarzda alohida munosabat bildirilishi lozim bo'lgan natijasi mana shunday hodisa bo'lib xizmat qilishi mumkin. Tarmoqlash operatori quyidagi formatga ega:

```
if ( {Shart} ) ... “u holda” bloki
[else
... “aks holda” bloki]
```

Shart – mantiqiy ifoda bo'lib, unga muvofiq interpretator qaysi blokni bajarish to'g'risida qaror qabul qiladi. Agar shart true («haqiqat») ifodasiga ega bo'lsa, “u holda” bloki bajariladi. Agar shart false («yolg'on») ifodasiga ega bo'lsa, “aks holda” bloki bajariladi (agar u mavjud bo'lsa). Bordiyu «aks holda» bloki mavjud bo'lmasa, dasturning navbatdagi ifodasi bajariladi. Agar shartning natijasi null yoki undefined bo'lsa, tarmoqlash operatori false ga qanday munosabat bildirgan bo'lsa, unga ham shunday munosabat bildiradi. Mana, bunga bir misol:

```
if (x == 1) { f = 3; h = 4; } else { f = 33; h = 44; }
```

Bu yerda biz x o'zgaruvchining ifodasini bir (1)ga taqqoslaymiz va taqqoslash natijalariga qarab f va h o'zgaruvchilarga har xil ifodalar beramiz. Shartga e'tibor bering – mantiqiy taqqoslash operatori shunday topiladi.

Taqqoslash operatorlari

6.1-jadval⁵³

Operator	Tavsifi	Operator	Tavsifi
<	Kichik	&&	Mantiqiy VA
>	Katta	//	Mantiqiy YoKI
==	Teng	!	Mantiqiy EMAS
<=	Kichik yoki teng	===	Qat'iy teng
>=	Katta yoki teng		
!=	Teng emas	! ==	Qat'iy teng emas

Dastlabki olti operator bilan hech qanday muammo tug'ilmasligi kerak – ular ikki ifodani bir-biriga taqqoslaydi va shart bajarilgan bo'lsa, true ni, bajarilgan bo'lmasa – false ni qaytaradi. Keyingi uch operator murakkabroq mantiqiy ifodalar tuzish imkonini beradi. Ular ikki mantiqiy ifoda ustida VA, YOKI va EMAS mantiqiy operatsiyalarini bajaradi va natijani qaytaradi.

Oxirgi ikki operator – «qat'iy teng» va «qat'iy teng emas» operatorlariga kelsak, ular qat'iy tenglik va qat'iy tengsizlik operatorlari deb ataladi. Gap shundaki, odatdagi «teng» va «teng emas» operatorlari har xil tipdagi operandlarga duch kelsa, ularni muayyan bir tipga aylantirishga harakat qiladi.

Qat'iy tenglik va qat'iy tengsizlik operatorlari bunday qilmaydi, operandlar o'zaro mos kelmagan taqdirda false ni qaytaradi. Bu ba'zan, masalan, foydalanuvchining kiritmasini tekshirishda foydalidir.

Biz yuqorida ko'rib chiqqan taqqoslash operatorlari har xil ustuvorlikka ega. Eng ustuvor operator - ! (mantiqiy EMAS) operatori. U ustuvorlik jihatidan inkrement, dekrement va inversiya operatorlariga teng. <, >, <(va >(operatorlari bitli siljish operatorlari orqasidan keladi, ularning orqasidan esa - ((, !(, (((va !(((operatorlari keladi. Ikkitalik YOKI ketidan mantiqiy VA, uning ketidan esa – mantiqiy YOKI keladi.

Ular eng past darajali ustuvorlikka ega.

⁵³Дротов В.А. JAVA SCRIPT в Web – дизайне. – СПб.: «БХВ-Петер-бург», 2002.- 880 с.

Operator ?

Bu yana bir shartli operator. Uning yozilish sintaksisi quyidagicha:

{Shart} ? {"u holda" ifodasi} : {"aks holda" ifodasi}

Bu operator shart haqiqiy bo'lgan taqdirda "u holda" ifodasini, shart haqiqiy bo'lmagan taqdirda esa – "aks holda" ifodasini qaytaradi.

$a = (f == 2) ? b : c + 2;$

Agar f o'zgaruvchi 2 ga teng bo'lsa, "a" o'zgaruvchiga "b" o'zgaruvchining ifodasini, aks holda esa " $c + 2$ " ifodasining natijasini joylashtiradi. Bu ustuvorlik darajasi eng past bo'lgan operatorlardan biridir. Undan quyida faqat qayd etish operatori turadi.

Davriy operatorlar

Davriy operator – bir necha marta bajariluvchi bloklar. Davrning takrorlanishi muayyan shart berilishiga qarab to'xtatiladi. Bu muayyan shart berilganida kodning muayyan bloki bajariladigan yoki bajarilmaydigan shartli operatorlar bilan bog'liq holatga o'xshaydi. JavaScript dasturchilarga davriy operatorlarning bir necha turlarini taklif qiladi[11,15].

FOR davriy operatori

Toki muayyan shart haqiqiy bo'lib qolar ekan, for davriyligi bajariladi (true qaytariladi). U dasturning muayyan fragmentini ma'lum marta bajarish uchun qo'llaniladi. Bu holda dasturning mazkur fragmenti necha marta bajarilganligini hisoblovchi butun son davriy operator hisoblagichi deb ataladigan o'zgaruvchiga joylashtiriladi. Dasturning davrda bajariluvchi bloki esa davriy operator tanasi hisoblanadi.

for ({Initsializatsiya ifodasi}; {Shart}; {Inkrement ifodasi})

... davriy operator tanasi

Initsializatsiya ifodasi o'zgaruvchi-hisoblagichning boshlang'ich ifodasini beradi. Shundan keyin shart tekshirib ko'riladi va agar uning ifodasi haqiqiy bo'lsa, davriy operator tanasi bajariladi. Shundan so'ng inkrement ifodasi hisoblagichning ifodasini o'zgartiradi va yana shart tekshirib ko'riladi. Toki shart yolg'on (false)

bo'lmagunicha bu amal qayta-qayta bajarilaveradi. Hisoblagich bilan bog'liq bunday davriy takrorlanishlar dasturlarda juda ko'p uchraydi.

```
for (i = 1; i < 11; i++)    { a += 3; b = i * 2 + 1; }
```

Bu davriy operator tanasi 10 marta bajariladi. Initsializatsiya ifodasi va shartga e'tibor bering. Biz i hisoblagichiga 1 boshlang'ich ifodasini beramiz va har sikl tanasi bajarilganidan keyin uni bittaga ko'paytiramiz. Hisoblagich ifodasi 11 ga etganida davriy operator tanasi bajarilishdan to'xtaydi va shart yolg'onga aylanadi.

Davriy operator tanasi blok ham bo'lishi mumkin. For operatoridan foydalanish misolini ko'ramiz:

```
for(i=0; i<document.links.length; i++)  
    { document.write(document.links[i].href+"<BR>"); }
```

Natija: [http:// ziyo.edu.uz/help/index.html](http://ziyo.edu.uz/help/index.html)

[http:// ziyo.edu.uz/help/shop.html#choice](http://ziyo.edu.uz/help/shop.html#choice)

[http:// ziyo.edu.uz/help/payment.html](http://ziyo.edu.uz/help/payment.html)

Operator , (vergul)

JavaScript asosan for davriy operatori inkrementining ifodalarida qo'llanuvchi operatorni taklif qiladi. Bu , (vergul) operatoridir. U inkrementning bir u, bir bu ifodasini ketma-ket bajarish imkonini beradi. U quyidagi formatga ega:

{“Toq ifoda”}, {“Juft ifoda”}

Davriylikning birinchi o'tishida «toq» ifoda, ikkinchi o'tishida – «juft» ifoda, uchinchi o'tishida esa – yana «toq» ifoda bajariladi va h.k.

```
for (i = 0; j < 11; i++, j++) {
```

```
a = i * 2 + 1; b = j / 2-3;}
```

, **(vergul)** operatori ustuvorlik jihatidan barcha operatorlar orasida eng oxirgi o'rinda turadi[15].

do-while davriy operatori

do-while davriy operatori for davriy operatoriga ko'p jihatdan o'xshash, biroq unda hisoblagich qo'llanilmaydi. Toki shart haqiqiy bo'lib qolar ekan, do-while davriy operator bajarilaveradi. Bunda shart davriy operator tanasi bajarilishidan oldin

emas, balki u bajarilganidan keyin tekshiriladi. Shu bois, hatto shart avvalboshdan soxta bo'lgan taqdirda ham, do-while davriy operatori hech bo'lmasa bir marta bajariladi.

do

... davriy operator tanasi

while ({Shart});

Siz do-while davriy operatorini har xil usulda qo'llashingiz mumkin:

```
do { a = a * i + 2; i = ++ i; } while (a < 100);
```

Yuqorida ko'rib chiqilgan misolda ajratilgan shartning berilishi tekshiriladi.

Operator while

While davriy operatori do-while davriy operatorini eslatadi, ammo shart davriy operator tanasi bajarilishidan oldin tekshiriladi. Shu bois, agar u avvalboshdan soxta bo'lsa, davriy operator biror marta ham bajarilmaydi.

Operator while davrni aniqlaydi. Umumiy holda quyidagicha aniqlanadi:

While (mantiqiy ifoda)

operator;

Operator va tuzilma (sostav) – davr tanasi hisoblanadi. Davr tanasi logik shartlar to'g'ri bo'lgandagina amalga oshiriladi:

```
while (flag==0)
```

```
{ id=setTimeout ("test();",500); }
```

Asosan bu tipdagi davr ba'zi hodisalarning davriy (periodicheskiy) harakatlarni bajarishdan oldin qo'llaniladi.

Misol uchun:

```
while (a < 100) { a = a * i + 2; i = ++ i; }
```

Operator for ... in

Berilgan operator ob'ekt xususiyatlarida “yugirib chiqish” imkoniyatini beradi. Misol:

```
for(v in window.document)
```

```
{ document.write(v+"<BR>"); }
```

break va continue operatorlari

Ba'zan tsiklning bajarilishini to'xtatish talab etiladi. Buning uchun JavaScript dasturchilarga break va continue operatorlarini taklif qiladi.

break operatori siklning bajarilishini to'xtatish va undan keyingi ifodaga o'tish imkonini beradi.

```
while (a < 100) {a = a * i + 2;  
if (a > 50) break; i = ++ i; }
```

Bu misolda biz, agar **a** o'zgaruvchining qiymati 50 dan oshmasa, davriy operatorning bajarilishini (ammo dasturning bajarilishini emas) to'xtatamiz.

continue operatori davriy operatorni qayta yuklash, ya'ni davriy operatorni tanasiga kiruvchi barcha keyingi komandalarni bajarmasdan qoldirish va uning boshiga qaytish imkonini beradi. Shart tekshirib ko'riladi (for tsikli, bundan tashqari, inkrement ifodasini bajaradi) va, agar shart haqiqiy bo'lsa, davriy operator tanasi boshidan bajariladi.

```
while (a < 100) {i = ++ i;  
if (i > a && i < 11) continue;  
a = a * i + 2;}
```

Bu yerda biz **i** ning 10 dan 20 gacha bo'lgan barcha ifodalari uchun **a** ning hisoblangan barcha ifodalarini o'tkazamiz.

break va continue operatorlarining to'liq formati quyidagi ko'rinishga ega:

break / continue [{Belgi}]

Bu yerda belgi ayni break yoki continue operatori qo'llanilayotgan tsiklni belgilaydi. Belgi quyidagi formatga ega:

{Belgi nomi} : ... Ifoda

Belgi nomi dastur doirasida betaktror bo'lishi lozim. O'zgaruvchilarning nomlariga nisbatan amal qiluvchi qoidalar belgi nomiga nisbatan ham amal qiladi. Belgilar Internet Explorer va Navigator ilovalarida 4.0 versiyalaridan boshlab qo'llanilmoqda. Bu dasturlarning oldingi versiyalarida break va continue operatorlari belgilar ko'rsatilmasdan qo'llanilgan.

Dasturga izoh

JavaScript ixtiyorimizga izoh kiritishning ikki operatorini beradi.

```
// ... Sharh satri
```

Bu operator ifoda oxiriga bir satrli izohni kiritish imkonini beradi.

```
a = b + c; // Bu – bir satrli izoh
```

E'tibor bergan bo'lsangiz, izoh ifoda tugaganini ko'rsatuvchi nuqta va vergul(;) dan keyin joylashgan.

```
/*
```

```
... izoh
```

```
*/
```

Bu operator esa dastur kodiga har qanday kattalikdagi izohni kiritish imkonini beradi.

```
/* Bu ifodada biz ikki o'zgaruvchining ifodasini qo'shamiz va natijani uchinchi o'zgaruvchiga joylashtiramiz.
```

```
*/
```

```
a = b + c;
```

§ 6.5. JavaScript dasturiy tili funktsiyalari

Dasturiy tillarda biror dastur kodidan ko'p marotaba foydalanish mexanizmi ishga tushishi hisobga olinishi kerak. Bunday mexanizm protseduralar va funktsiyalar bilan ta'minlanadi. JavaScript tilida *funktsiya* asosiy ma'lumotlar turidan biri sifatida yuzaga chiqadi. Bir vaqtning o'zida JavaScript tili bilan birgalikda ob'ekt Function ham aniqlangandir[15]. Funktsiya – bu JavaScript kodining maxsus yozilgan va rasmiylashtirilgan fragmenti bo'lib, uni dasturning istalgan joyidan chaqirish mumkin. Funktsiya betakror nomga ega bo'ladi, bir yoki bir nechta parametrlarni qabul qilishi mumkin va dasturda foydalanishga yo'l qo'yilgan natijani qaytaradi. Qoida tariqasida, kodning ko'p qo'llaniladigan muayyan fragmenti funktsiya tarzida rasmiylashtiriladi; undan dasturning turli joylariga nusxa ko'chirish o'rniga, funktsiyani yozgan va zarur holda uni turli parametrlar bilan chaqirgan ma'qul. Kodning funktsiyaga joylashtirilgan bunday fragmenti funktsiya

tanasi deb ataladi. Funktsiyani amalda qo'llashdan oldin uni e'lon qilish kerak. Bu **function** operatori yordamida amalga oshiriladi.

```
function{Funktsiya nomi }  
    ([{Vergullar bilan ajratilgan argumentlar ro'yxati}])  
    ... Funktsiya tanasi;
```

Funktsiya nomi dastur doirasida betakror bo'lishi kerak. O'zgaruvchilar va belgilarning nomlariga nisbatan amal qiluvchi qoidalar funktsiyalarning nomlariga nisbatan ham amal qiladi. Argumentlar ro'yxati funktsiyalarni chaqirish chog'ida kerakli argumentlar joylashtiriluvchi o'zgaruvchilar ro'yxatidir. Siz bu o'zgaruvchilarga qanday nom qo'ymang, ular baribir faqat funktsiya tanasi ichida qo'llaniladi. Bular funktsiyaning formal argumentlaridir. Funktsiya argumentlarining nomlari yoysimon qavs ichiga olinadi va vergullar bilan ajratiladi. Alohida holda argumentlar ro'yxati bo'lmasligi mumkin, biroq qavslar qolishi lozim. Funktsiya tanasi argumentlar ustidan muayyan amallarni bajaradi va natijaga erishadi. Uni funktsiyadan chaqiruv joyiga qaytarish uchun **return** operatori qo'llanadi:

```
return {O'zgaruvchi yoki ifoda };
```

Bu yerda o'zgaruvchi qaytarilayotgan ifodaga ega bo'lishi, ifoda esa uni hisoblashi lozim. Funktsiyani chaqirish uchun quyidagi format qo'llanadi:

```
{Funktsiya nomi}  
    ([{Vergullar bilan ajratilgan amaldagi argumentlar ro'yxati}])
```

Bu yerda funktsiya nomi ko'rsatiladi (u albatta betakror bo'lishi kerak) va yoysimon qavs ichida funktsiyani e'lon qilishda qo'llanilgan formal argumentlar emas, balki ustida tegishli amallarni bajarish lozim bo'lgan amaldagi argumentlar keltiriladi. Funktsiya o'zgaruvchiga berish yoki murakkab ifodada qo'llash mumkin bo'lgan natijani qaytaradi. Endi bunga misolni ko'rib chiqamiz.

```
function samplefunc (a, b)  
{var c; c = (a + b) / 2; return c; }  
.  
.  
.  
.  
.  
.  
var arg 1 = 1, arg 2 = 2, result;
```

```
result = samplefunc (arg 1, arg 2);
```

Bu yerda biz funktsiyani ikki son yig'indisining yarmini qaytaruvchi samplefunc nomi bilan belgiladik. Funktsiyani belgilashda biz argumentlarni qanday nomlaganimizga e'tibor bering. Shundan keyin biz bu funktsiyani dastur kodida chaqirdik va unga amalda ikkita argumentni uzatdik. Quyidagi farqni eslab qoling: funktsiyani e'lon qilishda formal argumentlar funktsiyaga dastur uzatadigan amaldagi argumentlar bilan bir emas. Formal argumentlar faqat funktsiyani e'lon qilish doirasida qo'llanadi. Funktsiyamiz natijasi return operatori tomonidan qaytarildi.

Funktsiyalarni bir-biridan ham chaqirish mumkin. Bunda funktsiyani chaqirishdan oldin uni belgilash lozimligini unutmang.

```
function samplefunc 1 (a, b)
{
. . . . .
}
function samplefunc 2 (c)
{
. . . . .
k = y + samplefunc 1 (x , 2);
}
```

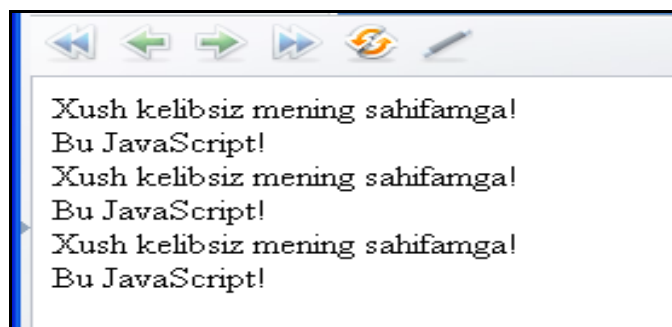
Endi global va lokal o'zgaruvchilar tushunchalarini ko'rib chiqamiz. Dastur kodida tashqi darajada, ya'ni funktsiyalarni belgilash doirasidan tashqarida e'lon qilingan o'zgaruvchilar global o'zgaruvchilardir. Ular hamma yoqdan ko'rinib turadi; dasturning istalgan joyidan, shu jumladan barcha funktsiyalarning belgilaridan hech bir muammosiz global o'zgaruvchiga murojaat etish, uning ifodasini olish yoki o'zgartirish mumkin.

Funktsiya tanasining tashqarisida emas, balki faqat uning ichida belgilangan o'zgaruvchilar lokal o'zgaruvchilardir. Dastur kodining qolgan qismi ularning mavjudligi haqida hech narsa bilmaydi. Lokal o'zgaruvchilar funktsiya chaqirilganida vujudga keladi, interpretator funktsiyani bajarishni yakunlab, dasturga qaytgach esa

ular tugatiladi deb hisoblash mumkin. Misol uchun matnni uch marta ketma-ket chop etuvchi skriptni ko'rib o'tamiz. Avval oson munosabatni ko'ramiz:

```
<html><script language="JavaScript">
<!-- hide
document.write("Xush kelibsiz mening sahifamga!<br>");
document.write("Bu JavaScript!<br>");
document.write("Xush kelibsiz mening sahifamga!<br>");
document.write("Bu JavaScript!<br>");
document.write("Xush kelibsiz mening sahifamga!<br>");
document.write(" Bu JavaScript!<br>");
// -->
</script>
</html>
```

Bu skript uch marta quyidagi matnni ekranga yozadi (6.4-rasm).



6.4-rasm. Uch marta ketma-ket chop etuvchi skript⁵⁴.

Agar yuqoridagi skriptga qarasak shu narsa ko'rinadiki, kerakli bo'lgan natijani olishda ushbu kodning ma'lum qismi uch marta takrorlangan edi. Function ob'ektidan foydalanib bu misolni mukammalroq yechish mumkin bo'ladi:

```
<html><script language="JavaScript">
<!-- hide
function myFunction()
{document.write ("Xush kelibsiz mening sahifamga!<br>");
  document.write ("Bu JavaScript!<br>");}
```

⁵⁴Дротов В.А. JAVA SCRIPT в Web – дизайне. – СПб.: «БХВ-Петер-бург», 2002.- 880 с.


```
myFunction(); myFunction(); myFunction();
// -->
</script></html>
```

Bu skript ham uch marta xuddi shu matnni ekranga yozadi. Ushbu skriptda aniqlangan funktsiya quyidagi satrdan iborat:

```
function myFunction()
{document.write("Xush kelibsiz mening sahifamga!<br>");
document.write ("Bu JavaScript!<br>"); }
```

Figuraviy qavslar - {} - ichidagi barcha skript buyruqlari *myFunction()* funktsiyasiga tegishli. Bunda har ikkala document.write() buyruqlari bir biri bilan bog‘liq va ko‘rsatilgan funktsiya chaqirib olinganda bajariladi. Misoldagi uch marta yozilgan myFunction() satrni jamlovchi ushbu funktsiya (figuraviy qavslar ichidagi ko‘rsatilgan buyruqlar) uch marta bajarilishini anglatadi. Ob’ektJavaScript funktsiya orqali aniqlanadi. Ob’ektni vujudga keltirish uchun o‘z navbatida Function orqali kiritiladigan konstruktordan foydalaniladi. Shu asnoda JavaScript tilida funktsiyalar keyingi savollar bilan bog‘liq bo‘ladi:

- funktsiya – ma’lumotlar turi;
- funktsiya – ob’ekt;
- ob’ekt konstruktorlari.

Ma’lumotlar turlari

Funktsiyani **function** kalit so‘zi yordamida aniqlanadi:

```
function f_name(arg1,arg2,...)
{ /* function body */ }
```

Bu yerda quyidagilarni e’tiborga olish kerak. Birinchidan, function funktsiyasi o‘zgaruvchi f_name ni aniqlaydi. Ushbu o‘zgaruvchi “function” turiga ega:

```
document.write("f_name o‘zgaruvchining turi:" +
typeof(f_name));
o‘zgaruvchi turi: f_name: function.
```

Ikkinchidan, bu o‘zgaruvchi quyidagi qiymatga ega bo‘ladi:

```
document.write("qiymat i:" + i.valueOf( ));
```

```
document.write("qiymat f_name:" + f_name.valueOf( ));
```

Ayni vaqtda `valueOf()` metodi (usuli) sanoqli o'zgaruvchi `i` va `f_name` larga qo'llaniladi. Shu sababli funktsiyaning qiymatini boshqa o'zgaruvchiga berish yo'li bilan funktsiyaning sinonimini belgilash mumkin:

```
function f_name(a)
```

```
{ if(a>=0) return true; else return false; }
```

```
document.write("O'zgaruvchi qiymati f_name:"+f_name(1)+"");
```

```
b = f_name;
```

```
document.write("O'zgaruvchi qiymati b:" + b(1) + "");
```

Natija: O'zgaruvchi qiymati `f_name:true`

O'zgaruvchi qiymati `b:true`

Ko'rinib turibdiki, agar funktsiyaning o'zgaruvchi o'zlashtirish mumkin bo'lsa, u holda boshqa argument funktsiyaning sifatida ham uzatish mumkin. Bularning barchasi **eval()** funktsiyasi foydalanilganda kuchayadi va bu JavaScript-kodini kechiktirib bajarilishini realizatsiya qilish imkoniyatini beradi. Kechiktirib bajarish – bu dasturlarni amalga oshirilish jarayonida uni o'zgartirish imkoniyatidir. Bir xil turdagi satr generatsiyasi evaziga kodning qisqarishi `eval()` xarakterli foydalanish hodisasidir:

```
for(i=0;i<5;i++) { eval("document.write('test'+i+'<br>')"); }
```

Natijasi: test0

test1

test2

test3

test4

Bevosita kodirovka qilishda beshta satr yozishga to'g'ri kelardi.

Massivlar

Massiv – massiv elementlari deb ataladigan bir turga mansub o'zgaruvchilarning raqamlangan to'plami. Bunda massiv elementining raqami indeks

deb, elementlarning umumiy soni esa – massiv miqdori deb ataladi. Massivni belgilash uchun maxsus massiv literali qo‘llaniladi. Massiv literali bu vergullar bilan ajratilgan elementlar ifodalarining kvadrat qavs ichiga olingan ro‘yxatidir.

```
var massive;
```

```
massive = [1, 2, 3, 4];
```

Bu yerda biz to‘rt elementdan iborat massivni belgiladik va ularga massive o‘zgaruvchisi nomini berdik. Shundan keyin biz elementlarning istalgan biriga uning indeksi yordamida kirishimiz mumkin. Buning uchun element indeksini massiv o‘zgaruvchisi nomidan keyin kvadrat qavs ichida ko‘rsatishimiz lozim.

```
a = massive [2];
```

Bu ifodada biz a o‘zgaruvchiga massiv uchinchi elementining ifodasini berdik (massiv elementlarini raqamlash noldan boshlanadi). Biz massivning ba'zi bir elementlarini belgilamasligimiz mumkin.

```
massive2 = [1, 2, , 4];
```

E’tibor bergan bo‘lsangiz, biz uchinchi elementni o‘tkazib yubordik va u raqamlanmay qoldi. Zarur holda biz massivga yana bir element kiritishimiz mumkin. Buning uchun unga kerakli ifodani berishimiz kifoyadir. Misol uchun:

```
massive [4] = 9;
```

Bunda massivning indeksi 4 va ifodasi 9 bo‘lgan yangi elementi yaratiladi (u hisob bo‘yicha beshinchi bo‘ladi). Hatto bunday qilish ham mumkin:

```
massive [7] = 9;
```

Bu holda to‘rtta yangi element yaratiladi va sakkizinchi elementning ifodasi 9 bo‘ladi. Beshinchi, oltinchi va ettinchi elementlar noaniq bo‘lib qoladi. Massivning istalgan elementini boshqa massiv (ichki massiv) bilan qayd etish mumkin.

```
massive2 [2] = ["n1", "n2", "n3"];
```

Shundan keyin siz ikkala indeksni: tashqi va ichki massivlarning indekslarini ko‘rsatib, ichki massivning istalgan elementiga kira olasiz.

```
str = massive2 [2] [1];
```

str o‘zgaruvchisi “n2” satriga ega bo‘ladi.

Funktsiya — ob'ekt

Har qanday JavaScript tili ma'lumotlar tipida ob'ektli "obertka" — Wrapper mavjud. U orqali ma'lumotlar tipi metodlarini o'zgaruvchi va literallarda qo'llashga sharoit yaratadi, shuningdek ularning xususiyatlari qiymatini qabul qiladi. Masalan, belgilarning satr uzunligi `length` xususiyatida aniqlanadi. Analogik "obertka" *ob'ekt* — `Function` funktsiyasida ham mavjud [14,15]. Masalan, funktsiya qiymatini nafaqat `valueOf()` metodi yordamida, balki `toString()` metodini qo'llashda ham ko'rish mumkin:

```
function f_name(x,y) {return x-y;}  
document.write(f_name.toString()+"<br>");
```

Komandaning natijasi:

```
functionf_name(x,y) { returnx-y; }
```

Funktsiya xususiyatlari dasturchi uchun faqat ularni funktsiya ichida chaqirilgandagina qo'llashi mumkin. Bunda asosan dasturchilar massiv funktsiya argumentlari (`arguments[]`), uning uzunligi (`length`), funktsiya nomi, chaqirib olingan funktsiya (`caller`) va (`prototype`) prototipi bilan ish olib boradi.

Funktsiya argumentlari ro'yhati va uzunligidan foydalanib, misolni ko'ramiz:

```
function my_sort( ) {a = new Array(my_sort.arguments.length);  
  for(i=0;i<my_sort.arguments.length;i++)  
    a[i] = my_sort.arguments[i]; b = a.sort(); return b;}  
b = my_sort(9,5,7,3,2);  
for(i=0;i<b.length;i++)  
  document.write("b["+i+"]="+b[i]+"<br>");
```

Natijasi:

b[0]=2

b[1]=3

b[2]=5

b[3]=7

Agar funktsiya boshqa funktsiyalar orqali chaqirilgan bo'lsa, u holda, callerxususiyatidanfoydalaniladi:

```
function slave( ) {document.write(slave.caller+"" ); return slave.caller;}  
function master1( ){ slave();}  
function master2( ){ slave();}  
.....  
master1( );  
master2( );
```

Oxirgi ikki qatorning amalga oshirilishining natijasi:

```
function master1( ) { slave( ); }  
function master2( ) { slave( ); }
```

Yana bir Function obekt xususiyati – bu prototype. Obektning faqat Function konstruktorini eslatib o'tamiz:

```
f = new Function(arg_1,...,arg_n, body)
```

Bu yerda f – Function sinf obekti. Shuningdek uni oddiy funktsiya misolida foydalanish mumkin. Konstruktor nomsiz funtsiyalarni qabul qilish uchun foydalanadi, bu esa ob'ekt metodlarini tayinlaydi yoki qayta aniqlaydi. Bu yerda biz ob'ektlarni qurish savollariga yaqinlashdik.

Ob'ekt tushunchasi

Ob'ekt – ko'plab o'zgaruvchilar – xossalarni va mazkur o'zgaruvchilarni manipulyatsiya qilishga mo'ljallangan funktsiyalar – metodlar to'plamini o'z ichiga olgan ma'lumotlarning murakkab tipi. Xossalar ma'lumotlarni saqlaydi, metodlar esa ularga ishlov beradi. Xullas, ob'ektga dasturning ancha murakkab amallarni bajaruvchi alohida, mustaqil fragmenti deb qarash mumkin.

Ob'ektlarga misol tariqasida HTML hujjatini ko'rib chiqamiz. U o'z xossalari to'plamiga ega: internet-adres, kattalik, simvollar kodi (agar u rus tilida yozilgan bo'lsa). Ayrim xossalarni o'zgartirish mumkin, ayrimlarini esa – mumkin emas. Shuningdek u metodlar to'plamiga ham ega: brouzer muloqot oynasisida ko'rsatish, diskda saqlash, chop etish. Siz kerakli metodlarni chaqirasiz va HTML hujjatining

ob'ekti amallarni qanday bajarishi haqida o'ylamaysiz: u bularning barchasini yaxshi bilishi nazarda tutiladi. Shundan kelib chiqib aytish mumkinki, xossalar bu ob'ekt atributlari, metodlar esa boshqarish vositalaridir.

Ob'ekt ma'lumotlarning boshqa tiplaridan yana shu bilan ham farq qiladiki, undan foydalanish uchun ob'ekt turkumiga mos bo'lgan nusxa yaratish talab etiladi. Bunda turkum deganda ob'ektning o'zgaruvchi tipiga o'xshagan o'ziga xos tipi, nusxa deganda esa – ishlash mumkin bo'lgan muayyan ob'ekt tushuniladi[12,15]. New operatori yordamida ob'ekt nusxasi yaratiladi; bunda o'zgaruvchiga ob'ektga ishora beriladi. Ishora (yoki ko'rsatgich) ob'ektning yaratilgan nusxasi joylashgan xotiradagi joyni qayd etadi. O'zgaruvchida saqlanayotgan ishora yordamida biz ob'ektga murojaat etishimiz mumkin.

```
var obj;
```

```
obj = new SomeObject ( );
```

Bu yerda obj – ob'ektning yaratilgan nusxasiga ishora beriladigan o'zgaruvchi, SomeObject – yaratiluvchi ob'ekt turkumi. Ob'ektdan foydalanib bo'linganidan keyin uni o'chirib tashlash mumkin. Buning uchun delete operatoridan foydalaniladi.

```
delete obj;
```

Ob'ekt xususiyatlariga kirish quyidagi chaqirish formati yordamida amalga oshiriladi:

```
{Ob'ektga ishorani saqlayotgan o'zgaruvchi nomi}. {Xususiyat nomi}
```

Shunday qilib, xususiyatga murojaat etish o'zgaruvchiga murojaat etishga o'xshash, faqat o'zgaruvchi-xususiyat nomining oldiga nuqta bilan ajratilgan o'zgaruvchi-ob'ekt nomini qo'yish kerak. Ma'lumki, xususiyat - ob'ektning bir qismi.

```
obj.SomeProperty = 9;
```

```
encoding = HTML Document.CodePage;
```

Metod ham deyarli shunday chaqiriladi:

```
{Ob'ektga ishorani saqlayotgan o'zgaruvchi nomi}. {Metod nomi} ([{Parametrlar}])
```

Ya'ni, ob'ekt nomini ko'rsatish shartini hisobga olmaganda, oddiy funktsiya bilan deyarli bir xil. Ob'ektni massiv elementiga berish va funktsiyaga uzatish

mumkin. Bunda funksiya tanasi doirasida uning xossalarini o'zgartirish va metodlarni chaqirishga ruxsat etiladi. Ob'ekt ob'ekt tipining xossalariga, ya'ni ichki ob'ektlarga ishoralarni saqlovchi xossalarga ham ega bo'lishi mumkin.

Ichki ob'ektlar

JavaScript tilida o'qish mumkin bo'lgan barcha ob'ektlar kiritilgan, foydalanuvchi tomonidan yaratilgan ob'ektlarga (foydalanuvchining ob'ektlariga) va boshqa dasturlardan olinadigan (tashqi) ob'ektlarga bo'linadi. DOM hosil qiluvchi ob'ektlar tashqi ob'ektlarga kiradi, chunki brouzer JavaScript interpretatoriga nisbatan tashqi dastur hisoblanadi. typeof operatori ob'ektlarning o'zgaruvchilariga "object" satrini qaytaradi.

Ob'ektlar iyerarxiyasi

HTML sahifasini interpretatsiya qilishda brouzer JavaScript ob'ektlarini yaratadi. Ob'ektlar ierarxik tuzilma shaklida saqlanadi va hujjatning tuzilishini aks ettiradi. Ierarxiyaning eng yuqori pog'onasida brouzer muloqot oynasisini ifoda etuvchi va qolgan barcha ob'ektlarning «yaratuvchisi» hisoblanuvchi window ob'ekti turadi. Ierarxiyaning quyi pog'onalarida joylashgan ob'ektlar o'zlariga bo'ysunuvchi ob'ektlarga ega bo'lishlari mumkin. Ikki tobe ob'ektga ega bo'lgan navigator ob'ekti alohida o'rin egallaydi. U brouzerning o'ziga tegishli bo'lib, uning xossalari ko'zdan kechirish dasturining xususiyatlarini belgilash imkonini beradi.

JavaScript ob'ektlarining kiritma turkumlari

JavaScript tilining yana bir o'ziga xos xususiyati shundaki, u ma'lumotlarning oddiy (sonli, satrli, mantiqiy) tiplariga ob'ektlar, ya'ni ob'ektlarning turkumlari deb qarashi mumkin. Kiritma ob'ektlar turkumlarining aksariyati ma'lumotlar oddiy tiplarining ob'ektda ifodalangan ko'rinishlaridir. Ularning ayrimlarini ko'rib chiqamiz.

Array massivi turkumi

JavaScript massivlarini ob'ektlar tarzida yaratish mumkin. Bunda konstruktorga bir yoki bir nechta parametrlar beriladi. Ma'lumki, konstruktor - ob'ektning maxsus metodi. Boshqa metodlardan u ob'ektni yaratishda chaqirilishi va

ob'ekt xossalarining boshlang'ich ifodalarini belgilashi bilan farq qiladi. Konstruktorning nomi doim ob'ekt turkumining nomi bilan mos keladi.

Shunday qilib, agar konstruktorga sonli tipga mansub bo'lgan bir parametr berilsa, elementlar tegishli miqdoridan iborat massiv yaratiladi. Tabiiyki, bu elementlar noma'lum bo'ladi. Agar konstruktorga o'zaro mos bo'lmagan tipdagi bir nechta parametr yoki bitta parametr berilsa, bu parametrlar massiv elementlari sifatida qo'llaniladi. Agar konstruktorga umuman hech qanday parametr berilgan bo'lmasa, nol uzunlikka ega, ya'ni elementlarsiz massiv yaratiladi[12,15].

```
arr = new Array (10);
```

```
arr = new Array (1, 2, 3, 4);
```

Boolean mantiqiy kattaligi turkumi

Mantiqiy kattaliklar ham ob'ekt ko'rinishida ifodalanishi mumkin. Bunga Boolean turkumi javob beradi. Boolean turkumi konstruktori parametr ko'rinishidagi mantiqiy ifodani qabul qiladi. Agar parametr o'tkazib yuborilgan bo'lsa, boshlang'ich ifoda true ga teng bo'ladi.

```
bool = new Boolean (f == q-1);
```

Boolean turkumi uch metodni qo'llab-quvvatlaydi. toSource metodi Boolean turkumining boshlang'ich kodi ifodalangan satrni qaytaradi. toString metodi ob'ektning "true" yoki "false" ifodasiga qarab satrli ko'rinishda qaytaradi. valueOf metodi – ob'ekt ifodasi: true yoki false.

Date sana turkumi

Sana turkumi sana va vaqt ifodasini saqlashga xizmat qiladi. Bu turkum konstruktori sana ifodasini sonli yoki satrli formatda qabul qiladi. Agar parametr sonli formatda ifodalangan bo'lsa, u 1970 yil 1 yanvar soat 00.00 dan keyin o'tgan milisekundlar soni sifatida talqin qilinadi. Agar u satrli formatda berilgan bo'lsa, konstruktor uni tegishli qoidalarga muvofiq sana ifodasiga o'tkazishga harakat qiladi.

Function funktsiya turkumi

Funktsiya turkumi funktsiyani ob'ekt singari manipulyatsiya qilish imkonini beradi. Ob'ekt – funktsiya konstruktori quyidagi ko'rinishga ega:

{Funktsiya nomi} (new Function ({Qavs ichiga olingan, vergullar bilan ajratilgan argumentlar ro'yxati} {Qavs ichiga olingan funktsiya tanasi}));

Bu demak, biz funktsiyalarni tanish usulda – function tayanch so'zi yordamida ham, ob'ektlar yaratiladigan usulda ham yaratishimiz mumkin. Ikkinchi holatda argumentlar ham, funktsiya tanasi ham qavs ichiga olinadi. Avval an'anaviy usulda, keyin yangi – ob'ekt usulida funktsiyani belgilashga misollarni ko'rib chiqamiz[15].

```
function samplefunc (a, b) { return (a + b)/2; }  
samplefunc = new Function ("a", "b", "(a + b)/2");
```

Bunda biz yaratilgan funktsiyani – u qaysi usulda yaratilganidan qat'i nazar, eskicha usulda chaqirishimiz mumkin.

```
f ( samplefunc (s, 2);
```

Funktsiya turkumi juda ko'p xossalar va metodlarni qo'llab-quvvatlaydi. Avval quyidagi muayyan foydalanuvchi obektidan sodda misolni ko'ramiz:

```
function Rectangle(a,b,c,d) {  
    this.x0 = a; this.y0 = b; this.x1 = c; this.y1 = d;  
    this.area = new Function  
    ("return Math.abs(this.x0-this.x1)* Math.abs(this.y0-this.y1)");  
    this.perimeter = new Function  
    ("return (Math.abs(this.x0-this.x1)+Math.abs(this.y0-this.y1))*2"); }  
c = new Rectangle(0,0,100,100);  
document.write(c.area());
```

Natija: 10000.

Bu misol funktsiya konstruktorlari illyustratsiya qilish uchun ishlatishda foydalanildi. rectangle()funktsiyasi – bu foydalanuvchi tomonidan aniqlanadigan Rectangle sinf ob'ektning konstruktoridir. Konstruktor ushbu sinfdan haqiqiy ob'ektni hosil qilishi mumkin. Funktsiya – bu ba'zi harakatlarni tavsifidir. Bu harakatlar bajarilishi uchun boshqaruv *funktsiyalarini* yuborish shart. Bu quyida keltirilgan misolda new operatori yordamida amalga oshiriladi. Bunda *funktsiya* ishga tushiriladi va huddi shunday haqiqiy ob'ekt hosil bo'ladi qilinadi[12,15].

To'rtta o'zgaruvchi hosil bo'ladi: `x0`, `y0`, `x1`, `y1` — bu `Rectangle` xususiyatidir. Bularga faqat keltirilgan sinfning kontekstdagi *ob'ekti* yo'l ochib berishi mumkin, masalan:

```
up_left_x = c.x0;
```

```
up_left_y = c.y0;
```

Xususiyatlardan tashqari yuqoridagi misolda konstruktor ichida ikkita `Function()` turidagi *ob'ekt* area va perimeter aniqlanadi. Bu berilgan sinfning *ob'ekt* metodlaridir. Ushbu *funktsiyalarni* faqat `Rectangle` *ob'ekti* sinfi kontekstida chaqirib olish mumkin:

```
sq = c.area();
```

```
length = c.perimeter();
```

Shunday qilib *ob'ekt* — bu xususiyatlar va metodlar to'plamidir.

Umuman olganda, o'zgaruvchilar funktsiya ichida uning xususiyatlari sifatida, metodlar sifatida esa funktsiyalarni ko'rib chiqish mumkin. Ayniqsa, yana bir kalit so'z — **this**ga e'tiboringizni qarating. U JavaScript-kod ayni vaqtda bajarilayotgan mazkur *ob'ektga* murojaat qilishga imkon beradi. Umuman olganda bu *ob'ekt* `Rectangle` sinfidandir.

Prototip

Asosan biz JavaScript ning tashkiliy *ob'ektlari* bilan ishlaymiz. Umuman esa, boshqa kurs bo'limlarida xususiyatlarga va tashkiliy *ob'ektlar* metodlariga bo'lgan munosabat yoritib berilgan. Prototype deb nomlanuvchi *ob'ektlar* xususiyatlari ma'no jihatdan juda qiziqarli. Prototip — bu aniq bir sinfning *ob'ekti* boshqa nomli konstruktoridir. Masalan, agar biz `String` sinfining *ob'ektiga* metodni qo'shishni xoxlasak:

```
String.prototype.out = new Function("a","a.write(this)");
```

```
...
```

```
"Salom!".out(document);
```

Natija: Salom!

Yangi metodni bayon qilishda String sinfining ob'ektlari uchun biz Functionkonstruktorini qo'lladik. Muhim bir farq bo'lib; yangi metodlarni va xususiyatlarni faqat prototip ob'ekti o'zgarishidan so'ng yuzaga keladigan ob'ektlar qo'lga kiritadi. JavaScript-dasturi boshqaruvni olgandan so'ng barcha tashkiliy ob'ektlar hosil qilinadi va bunda prototype xususiyatlari qo'llanilishida muhim bo'lgan cheklanishlar bo'ladi.

Shunday bo'lsa ham qanday qilib tuzilgan JavaScript sinfi ob'ektlariga metodlarni qo'shishni ko'rsatamiz. Tuzilgan nomlanish(vstroenny poimenovanny) Image misol bo'lib hizmat qiladi. Masala shundan iboratki, bunda URL rasmlarni huddi Link sinfining URL ob'ekti kabi terib chiqish kerak bo'ladi:

```
function prot(){a = this.src.split(':');protocol = a[0]+":";return protocol;}  
function host(){a = this.src.split(':');path = a[1].split('/');return path[2];}  
function pathname( )
```

```
{a = this.src.split(':');path = a[1].split('/');b = new array();  
for(i=3;i<path.length;i++) b[i-3]="path[i];<br" c = b.join('/');  
return "/" +c;}
```

```
Image.prototype.protocol = prot;
```

```
Image.prototype.host = host;
```

```
Image.prototype.pathname = pathname;
```

```
...
```

```
document.write("<IMG NAME=i1 SRC='image1.gif'><BR>");
```

```
document.write(document.i1.src+"<BR>");
```

```
document.write(document.i1.protocol()+"<BR>");
```

```
document.write(document.i1.host()+"<BR>");
```

```
document.write(document.i1.pathname()+"<BR>");
```

Asosiy g'oya shundaki konstruktor ishlatilishidan avvalroq qayta aniqlanishi (pereopredelit) kerak bo'ladi. HTML – panser JavaScript-interpretator ishga tushirishdan avval HTML qismlarga ajratadi va tuzilgan ob'ektlar hosil qilinadi. Bundan ko'rinib turibdiki sahifadagi *ob'ektni* JavaScript-kod orqali hosil qilish

kerak. Bunday holatda oldin *Image ob'ektning* qayta aniqlanishi sodir bo'ladi keyin esa, berilgan sinfnig tuzilgan ob'ekti hosil bo'ladi.

Object ob'ektning metodlari. Hujjatning ob'ekt modeli

Yana o'z skriptimizni ko'rib chiqamiz.

```
var d;
```

```
d ( new Date ( );
```

```
document.write (d.toString ( ));
```

document hujjati – umuman bizning hujjatimiz, write esa – uning metodi. Bu metod parametr sifatida berilgan matnni HTML hujjatining tegishli joyiga kiritadi.

Web-sahifani tavsiflovchi ob'ektlar, ularning xossalari va metodlari majmui hujjatning ob'ekt modeli (Document Object Model, DOM) deb ataladi. Web-sahifani yaratish, ya'ni HTML kodini JavaScript kodi bilan birlashtirish texnologiyasi (bunda JavaScript kodi ob'ekt modeli yordamida sahifani boshqaradi) dinamik HTML (Dynamic HTML) deb ataladi. Ilgari Microsoft va Netscape ob'ekt modelini amalga oshirish uchun taklif qilgan odatdagi HTML kengayishlari majmui ham dinamik HTML deb atalar edi. W3S 1 darajali DOM spetsifikatsiyasi (DOM level 1)ni qabul qilganidan keyin bu qo'shimchalarning barchasi HTML 4.0 ning oxirgi tahririga kiritildi.

Hodisalarga ishlov berish

Tizimda muayyan hodisa foydalanuvchining ma'lum harakati yoki tizim ichida muayyan shartning yuzaga kelishi natijasida sodir bo'lishi mumkin. Bizning holatda hodisa foydalanuvchi «sichqon» tugmasini bosishi, browser muloqot oynasining kattaligini o'zgartirishi, klavishni bosishi va sahifa yuklanishining tugallanishi natijasida sodir bo'lishi mumkin. Hodisaga shunday ta'rif berish mumkin: bu operatsion tizim yoki amaliy dastur shakllantiruvchi va «nimadir bo'ldi» degan ma'noni bildiruvchi signaldir. Endi hujjat (document) ob'ektida bir nechta xossalarni nazarda tutilgan bo'lib, ularga muayyan hodisa sodir bo'lganida bajariluvchi funktsiya ko'rsatkichlarini berish, maxsus kelishuvlardan foydalanib, kerakli hodisalarga javob beruvchi funktsiyalarni yozish va ularni mana shu xossalarga

berish mumkin deb faraz qilaylik. Endi joriy sanani aks ettiruvchi Web-sahifaga misolni ko‘rib chiqamiz.

```
<HTML><HEAD><TITLE> Bugun </TITLE><SCRIPT>
function writeDate ( ) { var d; d ( new Date ( );
    dateishere.innerText ( d.toLocaleString ( ); }
</SCRIPT></HEAD><BODY onload ( “writeDate ( )”>
<P ID ( “ dateishere ”> Bu yerda sana </P> bo‘ladi
</BODY></HTML>
```

Bu misolda hodisaga ishlov berish qo‘llanilgan:

```
<BODY onload ( “writeDate ( )”>
```

Bu onload hodisasiga writeDate ishlov beruvchisining funktsiyasini berish demakdir. Biz sahifa yuklanganidan keyin yuzaga keluvchi onload hodisaga ishlov beruvchisini yuklash uchun onload «soxta» atributidan foydalandik.

Boshqa bir misol:

```
<HTML><HEAD><TITLE>Bugun</TITLE><SCRIPT>
window.onload = writeDate;
function writeDate ( ) {var d;d = new Date ( );
dateishere.innerText = d.toLocaleString ( ); }
</SCRIPT></HEAD><BODY>
<P ID = “ dateishere ”>Bu yerdasana</P>Bo‘ladi
</BODY></HTML>
```

Bu yerda biz muloqot oynasi ob'ektining onload hodisasiga writedate funktsiyasi ko‘rsatgichini berdik va shu tariqa u mazkur hodisa sodir bo‘lganida bajarilishini qayd etdik. Xuddi shu tarzda hodisaga ishlov beruvchini sahifaning boshqa elementiga, masalan, rasmga bog‘lash mumkin:

```
<IMG SRC ( “Someimage.gif”
onlick ( “document. location.href ( ‘somepage.htm’;”>
```

Bu yerda biz rasmning onclick hodisasiga foydalanuvchini boshqa sahifaga yo‘naltiruvchi ishlov beruvchini berdik. Ya'ni biz HTML yordamida emas, balki

DOM va JavaScript vositasi yordamida giper ishora analogini yaratdik.

Biz hodisa har xil sabablarga ko'ra sodir bo'lishi mumkinligini qayd etib o'tdik. Quyida biz bunday sabablarning ayrimlarini sanab o'tamiz va ularga qisqacha tavsif beramiz:

1. Sahifaning muayyan elementiga yoki sahifaning o'ziga «sichqon» tugmasini bosish. Boshqa sahifaga o'tish (giper ishoralarni qo'llash analogi), muayyan izohni kiritish (biz oldingi bobda ko'rib chiqqan misollardan birida giper ishoralarga «sichqon» tugmasini bosib, qisqacha izohlarni kiritganga o'xshash) uchun qo'llanishi mumkin.

2. Klaviatura klavishini bosish. Qo'llash mumkin bo'lgan sohalar oldingi holatdagi bilan bir xil.

3. Sahifa elementlari yoki sahifaning o'zi ustidagi chiziqni siljitish. Ko'pincha giper ishoralarning «qaynoq» yoki «ta'sirchan» tasvirlarini yaratish uchun (ingliz Web-dizaynerlari ularni “rollover image” yoki “rollevers” deb ataydilar) va boshqa juda ko'p holatlarda qo'llanadi.

4. Web-sahifa yoki tasvir fayli yuklanishini yakunlash. Biz bu hodisadan turli boshlang'ich o'rnatmalar uchun foydalanganmiz.

Albatta, sahifa yoki ob'ektlar elementlarining muayyan turlariga xos bo'lgan boshqa hodisalar ham mavjud. Object — bu *ob'ekt* va unda metodlar ham bo'lishi mumkin. Asosiy metodlarni ko'rib chiqamiz: toString(), valueOf(), assign().

toString() metodi simvollar satrida ob'ektlarni hosil qilishni ta'minlaydi. Bu metod JavaScript – dasturlarida hamma erda qo'llaniladi, lekin aniq emas. Masalan, sonli yoki satrli ob'ektlarni chiqarishda. Misol, toString()ni funktsiyalarda qo'llash:

```
document.write(prot.toString()+"<BR>");
```

Natija:

```
function prot( )
```

```
{ a = this.src.split(':'); protocol = a[0]+":"; return protocol; }
```

Bu yerda prot()prototip misolidagi *funktsiya* qo'llanilgan. Agar huddi shunday Image *ob'ektni* chop etsak, u xolda rasm:[object] yuz beradi. Har doim ham toString() metodi ob'ektning satrli ekvivalenti mundarijasini qaytaravermaydi. U

shunchaki uning tipini qaytarib berishi mumkin. `valueOf()`metodi ham analogik tarzda o'zini namoyon qiladi. Bu metod ob'ektning qiymatini qabul qilishda imkon yaratadi. Ko'p hollarda bu metod `toString()` metodiga o'xshab ishlaydi[15]. Ayniqsa, agar qiymatni sahifaga chiqarish kerak bo'lsa:

```
document.write(prot.valueOf()+"<BR>");
```

Natija:

```
function prot() { a = this.src.split(':'); protocol = a[0]+":";return protocol; }
```

Ko'rinib turibdiki, huddi `toString()`metodi kabi natija olindi.

Yuqoridagi ikkala metodlardan farqli o'laroq `assign()`o'qishni emas, balki ob'ektlarning xususiyati va metodlarini qayta tayinlashga imkon beradi. Berilgan metod ba'zi qiymatlarni ob'ektga o'zlashtirish kontekstida foydalaniladi:

```
object = value; <=> object.assign(value);
```

Rassmotrim primer:

```
c = new Image();
```

```
c.src = "image1.gif";
```

```
b =new Image();
```

```
b.src = "image2.gif";
```

```
Image.prototype.assign = new Function("a","this.src = a.src;");
```

...

```
<A HREF='javascript:window.document.main.document.i2.assign(c);
```

```
void(0);'>Rasmni bo'yang</A>
```

Ushbu misolda `assign()`metodi JavaScript-kodi ichida emas, balki oddiy HTML-razmetkasida tatbiq qilinadi. Shunday qilib, metodni tatbiq etishdan avval uning o'z funktsiyasi qayta aniqlandandi.

6-bob bo'yicha xulosalar

6-bobda sahifalarni klient tomonidan boshqarish tili JavaScript haqida batafsil ma'lumot berilgan. HTML hujjatda JavaScriptdan foydalanish, `<SCRIPT>` tegi, JavaScript dasturida izohlardan foydalanish, ma'lumotlarni kiritish va natijalarni chiqarish usullari haqida ma'lumot berilgan.

Bundan tashqari ushbu bobda Java dasturlash tili, CGI texnologiyasi, SSI texnologiyasi, ASP texnologiyasi, VBScript dasturlash tili, Macromedia Flash kabi web-sahifa tuzishda foydalaniladigan dasturiy vositalar haqida qisqacha ma'lumotlar berilgan. Ob'ektga yo'naltirilgan dasturlash texnologiyalari, JavaScript tilida ob'ektlarning ierarxiyasi mavzularida zamonaviy dasturlash texnologiyalari haqida batafsil tushuncha berilgan. JavaScriptni ishga tushirish va uni HTML dasturida joylashtirish, xususiyatlar, usullar, hodisalar, o'zgaruvchilar, ma'lumotlar turlari, o'zgaruvchilarga boshlang'ich qiymat berish, global va lokal o'zgaruvchilar mavzulari orqali JavaScript dasturlash tilining asosiy g'oyasi ifodalangan. Ushbu bobda JavaScriptning zaxiralangan kalit so'zlari keltirilgan va ular bo'yicha ma'lumot berilgan. Arifmetik, qiymat berish va mantiqiy operatorlar, jarayonlarning bajarilish prioriteti mavzularida JavaScriptning arifmetik amallari va ularning bajarilishi haqida batafsil ma'lumot berilgan. JavaScript tilining asosiy operatorlari, JavaScript tilining funktsiyalari mavzularida keltirilgan dasturlar orqali operatorlar va funktsiyalardan foydalanish imkoniyatlari yoritib berilgan. Biriktirilgan ob'ektlar, Date va Boolean ob'ektlari, JavaScript dasturiy tilida muloqot oynasini boshqarish mavzularida JavaScript tilining standart ob'ektlari haqida bayon qilingan. Dasturlash texnologiyasining kuchli vositalaridan bo'lgan massivlar har bir dasturlash tilida o'ziga xos imkoniyatlarga egadir. JavaScript tilida massivlardan foydalanish bo'yicha ko'plab dasturlar misol sifatida keltirilgan.

Tayanch iboralar

Dinamik web-sahifa, method, atribut, JavaScript, script, Netscape, Sun Microsystems, ECMA (European Computer Manufacturers Association), Java, applet, CGI (Common Gateway Interface), SSI (Server Side Includes), ASP (Active Server Pages), VBScript, Dinamik HTML (Dynamic HTML), Active-X, ob'ekt, document.

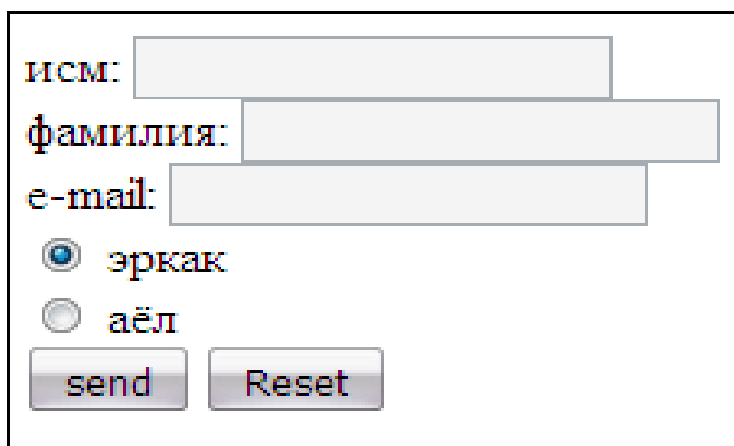
6-bob bo'yicha nazorat savollari

1. Dinamik sahifa deganda nimani tushunasiz ?
2. JavaScript tilining vazifalari nimadan iborat ?

3. Scriptqaysi teg yordamida ifodalanadi ?
4. Qanday dasturlash tillarini bilasiz ?
5. Script tegi vazifasi nimadan iborat ?
6. JavaScript tiliningqaysi operatorlarini bilasiz ?
7. JavaScript tilining funktsiyalari haqida nimalar bilasiz ?

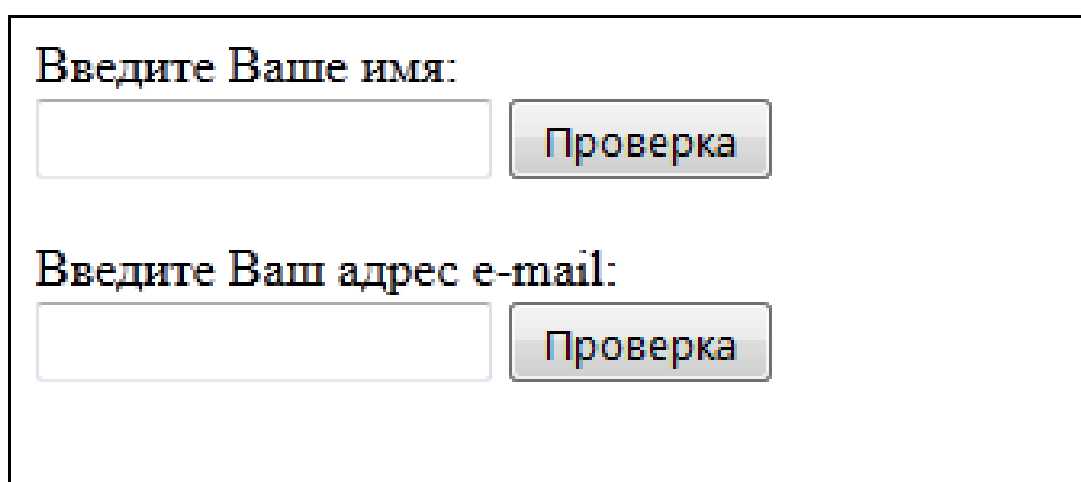
6-bob bo'yicha topshiriqlar

1. Quidagi JavaScript dasturini tuzing:



ИсМ:
фамилия:
e-mail:
☒ эркак
☐ аёл

2. Quidagi
JavaScript dasturini tuzing:



Введите Ваше имя:

Введите Ваш адрес e-mail: